

Cloud SQL Database Upgrade 2019 -2022

APPROACH 1: Using In Place Major Version Upgrade option from Google cloud Console

- Complexity of the Approach : Simple
- Downtime involved in the approach : Maximum
- Control over the upgrade : N/a , its google managed

IAM roles required for the upgrade and clone of instances are :

- roles/cloudsql.instances.clone
- roles/cloudsql.admin → it contains all Cloud SQL administrative permissions needed for cloning and upgrade related operations because the upgrade operation modifies the instance configuration/state.

IAM CLOUD SQL least permissions :

- cloudsql.instances.create
- cloudsql.instances.restoreBackup
- cloudsql.backupRuns.create
- cloudsql.instances.get
- cloudsql.backupRuns.list

→ Follow below steps from Google Cloud console page to upgrade the version of database.

1. In the Google Cloud console, go to the **Cloud SQL Instances** page.

[Go to Cloud SQL Instances](#)

2. To open the **Overview** page of an instance, click the instance name.
3. Click **Edit**.
4. In the **Instance info** section, click the **Upgrade** button and confirm that you want to go to the upgrade page.
5. On the **Choose a database version** page, click the **Database version for upgrade** list and select one of the available database major versions.
6. Click **Continue**.
7. In the **Instance ID** box, enter the name of the instance and then click the **Start upgrade** button.

The operation takes several minutes to complete.

Verify that the upgraded database major version appears below the instance name on the instance **Overview** page.

or you can use below Terraform script: → Set the version of upgrade according to the requirement

```
resource "google_sql_database_instance" "default" {
  name            = "sqlserver-instance"
  region          = "us-central1"
  database_version = "SQLSERVER_2022_ENTERPRISE"
  root_password   = "INSERT-PASSWORD-HERE"
  settings {
    tier = "db-custom-2-7680"
  }
}
```

Once the upgrade is completed the database would still be running under 2019 compatibility (150), that need to be manually changed at instance level using below T-SQL query : (THIS HAS TO BE DONE AFTER databases have been upgraded to IFM 11)

ALTER DATABASE <db_name> SET COMPATIBILITY_LEVEL = 160;

FAQs:

How long does an upgrade take?

If the instance hosts too many databases or tables, or your databases are very large, then the upgrade might take hours or even time out because the total upgrade time corresponds to the number of objects in the databases. If you have multiple instances that need to be upgraded, then your upgrade time increases proportionately.

Can we monitor each step in my upgrade process?

While Cloud SQL lets you monitor whether an upgrade operation is still in progress, you can't track the individual steps in each upgrade.

Can we cancel my upgrade after I've started it?

No, you can't cancel an upgrade once it has started. If your upgrade fails, Cloud SQL automatically recovers your instance to the previous version.

PFB below documentation with basic steps and terraform scripts provided by google :

<https://docs.cloud.google.com/sql/docs/sqlserver/upgrade-major-db-version-inplace>

Steps to be followed if clone role is missing:

Downtime involved : Downtime should be calculated from last On demand backup till the upgrade process completion.

Delta should be applied from application end , as the backup taken to replicate the databases is a snapshot backup and subsequent logs cant be applied on that to restore the delta transactions .

1. Take an on demand backup of the existing dev mpp instance backup
2. Create an empty new SQL 2019 instance using terraform (dev-mpp-clone)
3. Restore the on demand backup taken from dev mpp on to the cloned instance.
4. Once the restore is complete collect some evidences on the database and system configurations using below queries :

SELECT * FROM sys.configurations

SELECT * FROM sys.databases;

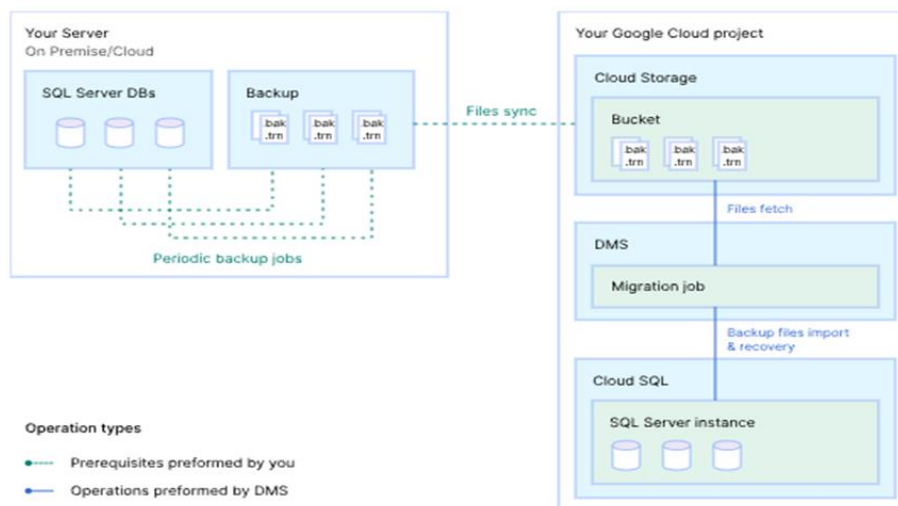
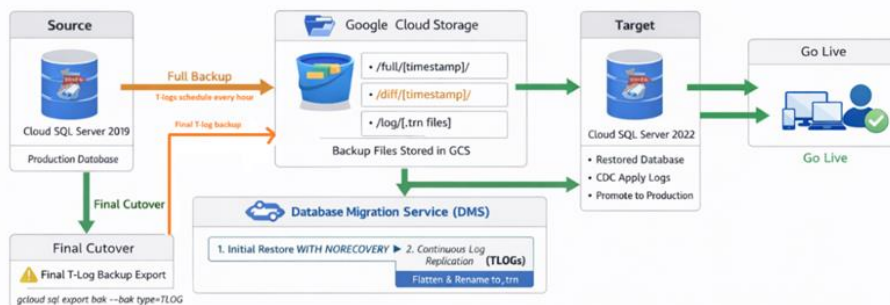
5. Then in place upgrade can be performed on the dev-mpp-clone using terraform script as stated in above steps. **(Approx time 10-15 mins)**
6. Once upgrade is completed validate the configurations value between dev-mpp (source) and dev-mpp-clone (Upgraded destination).
Configurations value from 2019 should be same in new upgraded instance with some additional parameters coming as a part of 2022 features ,
For example: 2022 instance will have query store enabled by default for any new database created.

APPROACH 2: SIDE by SIDE upgrade for Cloud SQL Server

This Approach involves Side by Side upgrade.

- Complexity of the Approach : Complex
- Downtime involved in the approach : 15-30 mins
- Control over the upgrade : Full Control over exporting backups and saving to the staging area (GCS) then restore will be done by DMS.

Please refer basic workflow diagrams for reference :



Overview :

This approach involves additional manual configuration and scripting activities as part of the migration workflow. Since Database Migration Service (DMS) requires backup files to be staged in a Google Cloud Storage (GCS) bucket before initiating the migration process, backups from the source Cloud SQL for SQL Server instance are first exported using gcloud commands and uploaded to the designated GCS bucket in accordance with Google Cloud recommendations and best practices. Once the backup files are available in the GCS bucket, DMS automates the restore process on the target Cloud SQL instance and continuously applies transaction log backups to keep the source and target environments synchronized until the final cutover activity is performed.

As it is a little complex workflow compared to approach 1 , so we would do it in phases to break the complexity :

Phase 1:

Create new SQL 2022 instance with same configuration as old SQL2019, same resources , CPU ,memory , storage , prefer do it via terraform script .

Phase 2 : Backup Export and GCS Configuration

Create a dedicated GCS bucket for DMS migration activities.
Grant the required Cloud SQL service accounts appropriate permissions on the bucket .
Export both FULL and Transaction Log backups to the GCS bucket in accordance with DMS requirements and Google Cloud guidelines.

Backup Export Automation

Backup export jobs will need to be automated for each database.

Example for database IFM_IDB:

One scheduled job for FULL backup export.
One recurring job scheduled every 15–30 minutes for Transaction Log backup exports.

These exported backup files will be continuously uploaded to the GCS bucket for DMS job consumption.

Phase 3 – DMS Migration Job Creation

Enable Database Migration API and Create the DMS migration job.

→ Source Instance type : SQL Server

→ Destination Instance Type : Cloud SQL for SQL server

Configure and validate Source connection profile and Destination connection profile

Perform connectivity validation and migration job testing prior to starting the migration.

Once initiated, the DMS job will continuously monitor the GCS bucket and automatically apply the latest Transaction Log backups to keep the target environment synchronized with the source environment.

Phase 4 – Final Cutover Activity

Once application downtime begins:

→ Verify the status of the Transaction Log backup export jobs.

→ Stop the Transaction Log backup jobs on the source environment.

→ Confirm that the latest Transaction Log backup has been successfully applied on the target instance through DMS.

→ Promote the migration job.

After promotion:

The new SQL Server 2022 instance becomes the active production instance. Application connectivity can then be redirected to the new environment.

Phase 5 – Login and SQL Agent Job Migration

This activity is independent of database migration and can be performed during earlier phases if required.

Activities include:

→ Extract logins from the source instance.

→ Extract SQL Agent jobs from the source instance.

Execute the generated scripts on the target SQL Server 2022 instance.

Phase 6 – Application Validation and Testing

Update the application connection string or endpoint to point to the new SQL Server 2022 instance.

Perform application validation testing.

Phase 7 – Database Compatibility Level Upgrade

Once the upgrade is completed the database would still be running under 2019 compatibility (150) , that need to be manually changed at instance level using below T-SQL query : (THIS HAS TO BE DONE AFTER databases have been upgraded to IFM 11)

ALTER DATABASE <db_name> SET COMPATIBILITY_LEVEL = 160;

IAM roles required :

- roles/cloudsql.admin → it contains all Cloud SQL administrative permissions needed for upgrade related operations
- roles/storage.admin → to create GCS bucket
- roles/iam.serviceAccountUser → to use the Cloud SQL service accounts

Important Permissions Applied on GCS Bucket to Source Cloud SQL service account

→ roles/storage.objectAdmin

→ roles/storage.legacyBucketWriter

Reason :

Write FULL backups to the GCS bucket

Continuously upload Transaction Log backups

Allow Cloud SQL export operations to create objects inside the bucket

Important Permissions Applied on GCS Bucket to Destination Cloud SQL service account

→ roles/storage.objectViewer

Reason :

Read FULL backup files from GCS

Read Transaction Log backup files

Continuously apply delta Transaction Log backups during replication